LiDAR Perception for Driverless Racing

Technical report: cone detection, track boundaries, racing line and odometry

E-Agle Formula Student Driverless perception task (LiDAR path) — October 2026

Victor Gabriele De Matteis

This report describes a LiDAR-only perception pipeline for a Formula Student Driverless car, written in C++20 with PCL, Eigen and NLopt, and what it achieves. From a single 24,000-point scan the pipeline finds the track cones, assigns them to the left or right boundary, and computes a minimum-curvature racing line. A second program estimates the car's motion between two scans with ICP. A Qt viewer shows every stage and lets an operator tune the parameters live. Because the recorded data has no labels, the work is validated against ground truth that is known by construction: synthetic tracks, synthetic motions applied to a real scan, and a ray-casting LiDAR simulator. Four automated tests run under `ctest`, and two evaluation tools write visual reports for human review. The validation found and fixed three defects that gave plausible-looking but wrong results. It also measured where the system stands: track-boundary and racing-line logic are correct on every clean test track, and odometry recovers known motions to 2 mm. The weak point is cone detection, with 37 % recall on simulated scans. The full pipeline needs 0.49 s per scan on one CPU core, against a 0.1 s real-time budget.

1 Summary

Key results (measured on an Apple M3, Release build)

Recorded scan (`cones.pcd`)	24,000 points → 67 clusters → 6 cones: 2 left, 3 right, 1 left unassigned
Left/right split, synthetic tracks	10/10 clean tracks correct (previous version: 2/10)
Racing line, known optima	centreline on a straight (error 0); outer edge on an arc (within 2.3 % of the half-width)
Odometry, known motions	error 2 mm and $< 0.01^\circ$ (previous settings: 0.77–0.96 m, $\approx 10^\circ$)
Cone detection, simulated scans	precision 79 %, recall 37 %, centroid error 6.9 cm
Runtime, cone pipeline	485 ± 22 ms per scan (2.1 Hz), one core
Runtime, odometry ICP	2.20 s per scan pair (median of 10)
Automated tests	4 `ctest` targets, all passing; 2 report generators

What works. The pipeline runs end to end on the recorded scan and produces a usable result. The geometry downstream of detection (boundary assignment, racing line, frame conventions, odometry transform) is verified against exact answers.

What does not, yet. Cone recall falls quickly with range, and the cone/non-cone decision does not separate cones from cone-sized objects. Section 6.5 quantifies both, together with a measured alternative. Throughput is about 5× below a 10 Hz sensor rate.

Maturity. A research prototype. It is validated in simulation and on one recorded scan, has regression tests for its geometric core, and has a clear, measured list of next steps (Section 8).

2 Scope and data

The task (`docs/task_brief.md`) asks for five levels on either camera or LiDAR data: load and preprocess, detect cones, classify cones versus obstacles, extract the track or racing line, and estimate odometry, plus an interactive debugging GUI. The LiDAR path was chosen. Levels 1–4 use `cones.pcd`; level 5 uses `first.pcd` and `second.pcd`.

Each scan holds 24,000 points. All x values are non-negative, so the sensor is forward-looking, and it sits about 0.82 m above the ground. The data contains no labels, no cone colours (intensity is not used) and no timestamps. After the loading transform, every module uses one frame: x **forward,** y **up,** z **right**; the track lies in the (x, z) plane. Every module was made to use this one convention; an earlier version mixed two conventions (Section 7).

3 System architecture

The repository builds two programs. `GUIAndRacingLine` is the Qt viewer that runs the cone pipeline; its logic lives in free functions under `logic/`, one file per stage. `OdometryWithProcessing` is a command-line program around a small odometry library (`odometry.h`). Tests and evaluation tools are in each project's `tests/` folder.

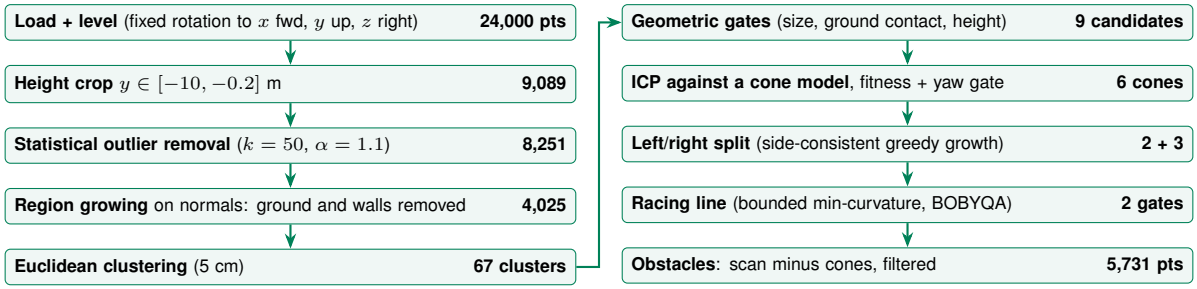


Figure 1: Cone pipeline on `cones.pcd` with the GUI default parameters, with the measured count after each stage. Odometry is a separate program (Section 6.4).

4 Methods

Preprocessing. A fixed rotation levels the scan. A height pass-through filter and statistical outlier removal (SOR) follow [7].

Ground and wall removal. Region growing on surface normals [5] finds smooth regions of at least 500 points. Cones never reach that size (the largest has 113 points), so every such region is removed.

Clustering. Euclidean cluster extraction with a 5 cm tolerance and 8–500 points per cluster [6].

Cone recognition. Clusters must have at least 10 points, touch the ground and be 0.2–0.5 m tall. Each survivor is aligned to a synthetic cone surface with point-to-point ICP [1] and accepted if the mean squared residual is below 0.005 m^2 and the rotation about the vertical axis is below 13°.

Left/right split. Seeds: the closest cone on each side of the car. Each boundary then grows by its nearest remaining cone that lies on the correct side of the other boundary and closer to its own, with the turn limited to $\pm 45°$. The two side conditions are mutually exclusive, so a boundary cannot jump across the track.

Racing line. Left and right cones are paired into gates. The decision variables are the lateral offsets d_i along each gate's normal, bounded by the half-width w_i. The cost is

$$J(\boldsymbol{d}) = \sum_i \kappa_i^2 + 0.1 \sum_i (d_{i+1} - d_i)^2, \qquad \kappa_i = \frac{2\,\theta_i}{\|\boldsymbol{a}_i\| + \|\boldsymbol{b}_i\|}, \qquad |d_i| \le w_i, \tag{1}$$

where $\theta_i = \operatorname{atan2}(\boldsymbol{a}_i \times \boldsymbol{b}_i,\ \boldsymbol{a}_i \cdot \boldsymbol{b}_i)$ is the turning angle between consecutive segments $\boldsymbol{a}_i, \boldsymbol{b}_i$. The car is a fixed first point, so the line leaves the car smoothly. The problem is solved with the derivative-free, bound-constrained BOBYQA [4] from NLopt [3].

Odometry. Both scans are cropped, levelled and SOR-filtered, then registered with point-to-point ICP (70 iterations, correspondences limited to 1 m). PCL returns T_{21}, which maps the first scan into the second. The car's displacement in the first frame is the translation of T_{21}^{-1}, i.e. $-R^\top \boldsymbol{t}$.

5 Validation approach

The recorded scans have no labels, so accuracy cannot be measured on them. Instead, every module is tested on inputs whose correct output is known by construction:

The two HTML reports draw every case from above: true and predicted sides, wrong cones circled, missed cones and false positives marked. A human reviewer can then confirm the metrics, and check the recorded scan, for which no metric exists.

6 Results

6.1 Recorded scan

Six cones are detected, with ICP residuals of 1.9–3.4 cm RMS. Two are assigned to the left boundary and three to the right. With two cones on the left there are two gates. The optimal line through them is a straight line from the car (cost ≈ 0), which is the expected minimum-curvature answer for such a short horizon.

6.2 Left/right split

When a decision is ambiguous, the current split leaves the cone unassigned rather than putting it on the wrong side. This keeps wrong boundaries out of the racing line.

Module	Ground truth	Tool
Left/right split	10 synthetic tracks (straights, curves, S-curve, hairpin, car offset and yawed), clean and with noise or missing cones, 100 random seeds each	`eval_left_right` (ctest + HTML report)
Racing line	Analytic optima (straight, constant arc), exact interpolation geometry, bounds	`test_racing_line` (ctest)
Cone ICP	Known rotations of the cone model; simulated single views of cones and cone-sized non-cones	`test_cone_icp` (ctest)
Odometry	A known motion applied to `first.pcd`, with independent sampling, 1 cm noise and partial overlap	`test_odometry` (ctest)
Whole cone pipeline	Ray-cast LiDAR scans (24,000 rays) of 5 scenes with cones, walls, obstacles and cone-sized decoys	`eval_pipeline` (HTML report)

Table 1: Validation matrix.

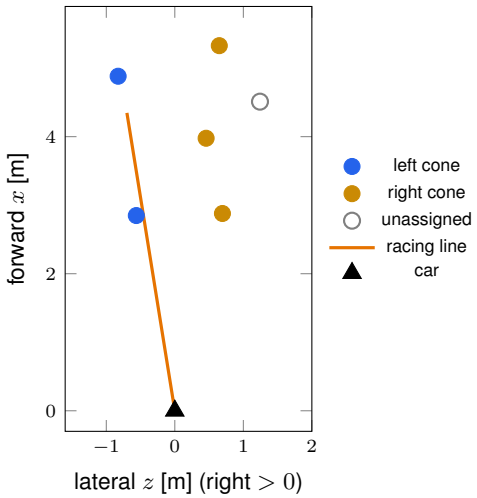


Figure 2: Result on `cones.pcd` (top view). The sixth cone sits 0.6 m outside the right boundary and fits neither side, so it is left unassigned rather than guessed.

Condition (10 tracks, 100 seeds each)	Previous version	Current
Clean tracks classified perfectly	2 / 10	**10 / 10**
Accuracy on clean tracks	7.7–100 %	**100 %**
Perfect runs, 5 cm position noise	0 %	**100 %**
Perfect runs, 15 cm position noise	0 %	68–100 %
Perfect runs, 10 % of cones missing	0–79 %	44–100 %

Table 2: Left/right split on synthetic tracks. Ranges are over the 10 tracks; the weakest cases are the hairpin and the S-curve with missing cones.

6.3 Racing line

- **Straight track:** the line is the centreline; maximum offset 0.000 m.
- **Constant arc ($R = 10$ m):** the line hugs the outer edge. The end points sit exactly on the bound and interior points within 2.3 % of the half-width. That small gap is the true discrete optimum: it has a lower cost than putting every point on the bound.
- **Start at the car:** including the car in the cost reduces the initial turn from 57.3° to 0.9°.
- **All 10 synthetic tracks:** no bound violation, no reversal, forward progress at every gate; largest turn 24.6°.
- **Drawn line:** it passes through every optimised point, in order, within $2\,\mu$m.

6.4 Odometry

	Unbounded (previous)		1 m bound (current)	
True motion (forward, left, yaw)	position err.	rotation err.	position err.	rotation err.
0.5 m, 0 m, 0°	0.837 m	10.07°	**0.002 m**	**0.00°**
0.5 m, +0.2 m, +2°	0.962 m	8.66°	**0.002 m**	**0.00°**
0.3 m, −0.1 m, −3°	0.873 m	10.13°	**0.002 m**	**0.00°**
1.0 m, 0 m, +5°	0.774 m	10.08°	**0.002 m**	**0.00°**

Table 3: Odometry on synthetic scan pairs built from `first.pcd` with a known motion. With PCL's default unbounded correspondences, points with no counterpart in the other scan dominate the fit; with a 1 m bound the motion is recovered.

On the recorded pair the current estimate is 0.41 m forward and 0.27 m left (0.49 m in the ground plane), with a 6.3° rotation. ICP stops at the 70-iteration cap, and the residual over matched pairs is 0.46 m RMS.

Limitation: the recorded odometry pair cannot be verified

The two recorded scans are levelled with different hand-tuned pitch corrections ($-10°$ and $-3°$). Part of the estimated rotation and the 0.21 m vertical motion most likely absorb that difference. The method is shown correct on synthetic pairs; the accuracy on this particular pair is unknown. Automatic levelling (ground-plane fit) and a motion prior would remove this uncertainty.

6.5 Cone detection

On simulated scans (5 scenes, 3 seeds), the detector reaches **79 % precision and 37 % recall**; the centroid error is 6.9 cm, biased towards the sensor because only the near side of each cone is seen. Tracing every true cone through the stages shows three separate losses:

1. SOR with $k = 50$ neighbours removes every cone beyond about 6 m: such cones have fewer than 50 points, so their neighbourhood reaches the ground and they look like outliers.
2. The 5 cm clustering tolerance starts splitting cones at about 5 m.
3. The yaw gate rejects true cones. A cone is symmetric about its axis, so ICP cannot measure its yaw: a cone turned 30° about its axis is reported at $-7.3°$. The gate accepts 85 % of simulated cone views, but also 95 % of cone-sized boxes and 100 % of buckets.

The tilt of the fitted axis is an observable quantity and separates the classes in simulation (cones 5.6–26.6°, non-cones 30.7–56.8°). Table 4 measures the effect of replacing the gate and retuning the two filters.

Detector variant	Precision	Recall	Sides ok	Line in track	Cones, recorded
Current (yaw $< 13°$)	79 %	37 %	85 %	59 %	6
No yaw gate	73 %	52 %	82 %	80 %	9
Tilt $< 20°$	94 %	48 %	98 %	88 %	7
Tilt $< 20°$, SOR $k = 10$, clustering 8 cm	96 %	70 %	75 %	88 %	6

Table 4: Detector variants on simulated scans, and the number of cones each finds on the recorded scan. "Line in track" is the share of the racing line inside the true track, where a line was produced.

The tilt gate is clearly better in simulation. On the recorded scan, two candidates rejected by the current gate, at $(x, z) = (1.78, +0.55)$ and $(1.99, -0.65)$ m, form a symmetric pair 1.2 m apart and about 1 m before the first detected

pair: very likely the nearest gate of cones, partly cut by the lower edge of the field of view. The tilt gate recovers one of them. Without labels this cannot be confirmed, so the production gate was left unchanged pending a visual check.

6.6 Runtime and resources

Stage	Time per scan	Notes
Cone detection (crop to ICP)	221 ± 5 ms	
Obstacle extraction	264 ± 18 ms	second normal estimation and region growing
Left/right split	0.002 ms	
Racing line	0.034 ms	
Cone pipeline total	$\mathbf{485 \pm 22}$ **ms**	median 474 ms; 2.1 Hz; peak memory 120 MiB
Odometry ICP	**2.20 s**	median of 10 runs; 0.45 Hz; 92 % of one core; 108 MiB

Table 5: Apple M3, macOS 15.6, Release build (`-O3`), single thread. Cone pipeline: 10 timed runs after a warm-up.

7 Defects found by the validation

Three defects produced results that looked plausible on the recorded data but were wrong. Only tests with known answers exposed them.

1. **Left/right split on one coordinate.** A coordinate swap put the lateral position in a field the algorithm did not read, so distances and angles used the lateral offset alone. Fixed by working in the ground plane throughout; 2/10 $\rightarrow$ 10/10 clean tracks.
2. **Racing line drawn differently from what was optimised.** Each drawn segment reused one offset for both ends, the last point was never drawn, and the optimiser ignored the car. The old curvature term also gave a U-turn zero cost. All four are fixed and covered by tests.
3. **Odometry off by about 0.8 m.** Unbounded correspondences, plus a sign inversion in the printed displacement. Fixed with a 1 m bound and a single, tested displacement formula.

8 Limitations and roadmap

Prio.	Item	Evidence and proposed action
1	Cone recall at range	37 % recall; SOR and clustering lose far cones. Make both range-aware, or downsample adaptively; target the 70 % measured with retuned filters.
2	Cone/non-cone decision	Yaw is not observable; replace it with a tilt gate after confirming the extra recorded cluster; add labelled scans for precision/recall by range.
3	Real-time throughput	0.49 s vs. 0.1 s budget. Share normals and kd-trees between the two region-growing passes, downsample, parallelise.
4	Odometry on real data	Automatic levelling (ground-plane RANSAC [2]), motion prior, point-to-plane ICP or GICP [8]; report convergence and inlier ratio.
5	Gate pairing	Gates pair the i-th left cone with the i-th right cone; a missed cone skews a gate. Pair by nearest opposite cone instead.
6	Code structure	Free functions with exported constants; move to classes with explicit parameters and ownership.

Table 6: Prioritised next steps, each tied to a measurement in this report.

9 Reproducing the results

Dependencies: PCL 1.15, VTK 9.5, Eigen, NLopt 2.10, Boost and Qt 5 or 6 (GUI only); CMake $\geq$ 3.5 (4.0 for the odometry project). From each project folder:

```
cmake -DCMAKE_BUILD_TYPE=Release -G Ninja -S . -B build && cmake --build build
ctest --test-dir build
```

The two visual reports are written by:

- `eval_left_right --pcd cones.pcd --model ideal_cone.ply` (left/right split);
- `eval_pipeline --model ideal_cone.ply` (whole cone pipeline on simulated scans).

The repository README lists every command.

The code footprint is 1,119 lines for the pipeline logic, 452 for the viewer, 239 for odometry, and 1,361 for tests and evaluation tools (non-blank lines).

References

[1] P. J. Besl and N. D. McKay. A method for registration of 3-D shapes. IEEE Transactions on Pattern Analysis and Machine Intelligence, 14(2):239–256, 1992.

[2] M. A. Fischler and R. C. Bolles. Random sample consensus: A paradigm for model fitting with applications to image analysis and automated cartography. Communications of the ACM, 24(6):381–395, 1981.

[3] S. G. Johnson. The NLopt nonlinear-optimization package. `https://github.com/stevengj/nlopt`, 2007.

[4] M. J. D. Powell. The BOBYQA algorithm for bound constrained optimization without derivatives. Technical Report NA2009/06, Department of Applied Mathematics and Theoretical Physics, University of Cambridge, 2009.

[5] T. Rabbani, F. A. van den Heuvel, and G. Vosselman. Segmentation of point clouds using smoothness constraint. In ISPRS Commission V Symposium, International Archives of Photogrammetry, Remote Sensing and Spatial Information Sciences, volume 36, pages 248–253, 2006.

[6] R. B. Rusu. Semantic 3D Object Maps for Everyday Manipulation in Human Living Environments. PhD thesis, Technische Universität München, 2009.

[7] R. B. Rusu, Z. C. Marton, N. Blodow, M. Dolha, and M. Beetz. Towards 3D point cloud based object maps for household environments. Robotics and Autonomous Systems, 56(11):927–941, 2008.

[8] A. Segal, D. Hähnel, and S. Thrun. Generalized-ICP. In Robotics: Science and Systems (RSS), 2009.